

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

RV COLLEGE OF ENGINEERING®

(Autonomous Institution affiliated to VTU, Belagavi)



PROJECT REPORT

On

LogiRoute: An Algorithmic Pipeline for Last-Mile Logistics

Submitted in fulfilment of Second year DAA Lab

Submitted by

- | | |
|-----------------------------|---------------------|
| 1. MANSI RAI | [1RV24CS149] |
| 2. K LAXMI SRAVANTHI | [1RV24CS133] |

Under the guidance of

**Dr. Ganashree K N
(Assistant Professor)**

2025 – 2026

RV COLLEGE OF ENGINEERING®, BENGALURU-560059
(Autonomous Institution Affiliated to VTU, Belagavi)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work titled “LogiRoute: An Algorithmic Pipeline for Last-Mile Logistics” carried out by Mansi Rai (1RV24CS149) and K Lakshmi Sravanthi (1RV24CS133), in partial fulfilment of the DAA Lab course introduced in the second, during the academic year 2026-2027. It is certified that all necessary corrections/suggestions indicated during internal assessment have been duly incorporated in the final report. The project work has been approved as it satisfies the academic requirements in respect of the DAA Lab Experiential Learning Project as prescribed by the institution.

Dr. Ganashree K N
Assistant Professor
Department of Computer
Science and Engineering

External Examiners

Name of Examiners

Signature with Date

1. _____

2. _____

ABSTRACT

In the contemporary logistics and ecommerce sectors, last-mile delivery represents the most financially volatile and operationally complex phase of the supply chain. This complexity stems from the intersection of multiple classical, computationally intensive bottlenecks: resource-constrained packing, dependency scheduling, global pathfinding, and optimal route sequencing. Because the routing phase scales factorially as an instance of the Travelling Salesman Problem (TSP), legacy brute-force paradigms fail due to the exponential explosion of the state-space tree ($O(n!)$), resulting in severe system latency and highly sub-optimal fleet utilization.

This project introduces **LogiRoute**, a modular, event-driven simulation framework designed to optimize last-mile logistics via a multi-stage **Algorithmic Pipeline**. The system abstracts and executes optimization across five interconnected layers:

1. **Selection Layer:** Employs a multi-dimensional (2D) 0/1 Knapsack algorithm utilizing Dynamic Programming (DP) tabulation to simultaneously maximize payload value across discrete weight (kg) and volume (L) vehicular capacities.
2. **Dependency Layer:** Resolves operational and physical package sorting constraints using Topological Sorting implemented via Depth First Search (DFS) on a Directed Acyclic Graph (DAG).
3. **Pathfinding Layer:** Computes an all-pairs shortest path matrix across a weighted, directed urban graph using the Floyd-Warshall algorithm, establishing an $O(1)$ spatial distance lookup table.
4. **Dynamic Phase Layer:** Introduces asynchronous graph re-weighting to mitigate live traffic disruptions or road blocks via localized matrix updates and partial re-routing, avoiding full-pipeline recalculation overheads.

Empirical evaluation demonstrates that while heuristic approximations offer immediate execution boundaries, the exact Branch and Bound framework achieve an average 20% to 25% reduction in total distance traversed compared to baseline greedy approaches. By transforming theoretical Design and Analysis of Algorithms (DAA) concepts into an integrated software pipeline, LogiRoute provides a scalable, computationally stable blueprint for reducing operational fuel margins, minimizing computational latency, and achieving sustainable fulfilment targets in modern logistics architectures.

TABLE OF CONTENTS

- Introduction
- Literature review
- System Design and Architecture
- Software Implementation
- Experimental Setup
- Results & Discussion
- Conclusion
- Appendices (Codes, Photos)

Chapter 1: Introduction

1.1 Background

The modern e-commerce and logistics sector has witnessed an unprecedented surge in transaction volume, making the "Last-Mile Delivery" phase, the final journey of a parcel from a local distribution hub to the customer's doorstep, the most critical link in the supply chain. Statistically, last-mile operations account for up to **41% to 53% of the total cost of fulfillment**, making it a primary point of financial volatility for global enterprises like Amazon and Flipkart, as well as fast-commerce platforms like Blinkit and Zepto.

Historically, logistics scheduling relied heavily on human dispatchers and driver intuition. However, human operators fail when forced to compute multi-variable constraints simultaneously. A human supervisor can rarely pack a vehicle to maximum mathematical weight and volumetric bounds, nor can a driver intuitively plan an optimal sequential path when the delivery list exceeds ten stops. In computer science, this intersection of loading optimization, graph theory, and path sequencing is recognized as a complex variation of the Multi-Constraint Vehicle Routing Problem (VRP).

LogiRoute serves as an experimental algorithmic platform built to demonstrate how an integrated, multi-stage pipeline of discrete mathematical algorithms can automate and systematically optimize these operations. By chaining together core concepts from the DAA syllabus, Dynamic Programming, Graph Analytics, Heuristics, and State-Space Search, the project establishes a computational framework that replaces human guesswork with mathematical proof.

1.2 Problem Statement

Legacy logistics management platforms and human-driven distribution workflows suffer from structural inefficiencies due to three interconnected, computationally intensive bottlenecks:

1. **The Multi-Dimensional Resource Bottleneck (Selection):** Fleet deployment is plagued by cargo space under-utilization, where vehicles run out of volumetric storage capacity long before hitting gross weight limits, or vice versa. Standard single-constraint models fail to optimize multi-dimensional variables (weight, volume, and package delivery

deadlines simultaneously), leading to a high percentage of unused payload capacity.

2. **The Topological Routing Bottleneck (Pathfinding):** Urban environments are highly dynamic, characterized by directed one-way grids, unpredictable traffic patterns, and sudden road closures. Legacy routing systems often use static, pre-calculated coordinate models that take hours to manually adjust when an incident occurs, creating substantial operational lag.
3. **The Combinatorial Explosion Bottleneck (Sequencing):** Determining the most fuel-efficient sequence of drop-offs for a delivery vehicle containing multiple packages is an instance of the **Travelling Salesman Problem (TSP)**, which is fundamentally **NP-Hard**. Because the search space scales factorially $O(n!)$, a vehicle tasked with a seemingly small number of stops (e.g., 15 houses) faces over **1.3 trillion possible routing configurations**. Blindly computing every path via Brute Force locks CPU threads and freezes application states, while unvalidated Greedy heuristics frequently yield sub-optimal routes that increase total fuel expenditure.

1.3 Objectives

The primary technical and academic objectives of **LogiRoute** are defined as follows:

- **Engine a Multi-Stage Algorithmic Pipeline:** To design a modular software system where the output of one algorithm seamlessly structures the input for the next, modelling a clean separation of concerns.
- **Maximize Vehicular Payload Utility:** To implement a multi-constraint (2D) 0/1 Knapsack optimization engine using Dynamic Programming tabulation, ensuring that truck fleet deployment achieves near-zero capacity wastage across both weight (kg) and volume (L) parameters.
- **Establish an Instantaneous Spatial Lookup System:** To construct a weighted, directed graph engine that utilizes the Floyd-Warshall algorithm to pre-compute an all-pairs shortest path matrix, reducing real-time routing calculation lookups to $O(1)$ constant time.

- **Mitigate Combinatorial Complexity:** To implement a deterministic Branch and Bound state-space search algorithm that utilizes bounding functions to prune non-optimal routing paths, resolving the TSP bottleneck without encountering factorial execution freezes.
- **Benchmark Algorithmic Paradigms:** To contrast the exact optimal results of Branch and Bound against a low-latency Nearest Neighbour Greedy heuristic, mapping out empirical data on the time-space trade-offs and approximation ratios.
- **Build an Event-Driven Self-Healing Engine:** To develop an asynchronous graph re-weighting mechanism capable of handling live road blocks or traffic incidents by performing localized matrix updates and partial re-routing, bypassing full system recalculations.

1.4 Scope of the project

What is Inside the Scope (Included):

- **Modular Multi-File Python Framework:** A strictly structured Python framework containing dedicated, decoupled modules for data models (`data_structures.py`), optimization logic (`algorithms.py`), graph states (`city_manager.py`), and a master executor pipeline (`simulator.py`).
- **Multi-Constraint Optimization Engine:** Full implementation of a 2D 0/1 Knapsack dynamic programming array, tracking subproblems across discrete item counts, weight steps, and volume intervals.
- **Dependency Resolution Engine:** DFS-based Topological Sorting on a Directed Acyclic Graph (DAG) to enforce strict package sorting rules (e.g., heavy parcels layered underneath fragile items).
- **Dual-TSP Solver Implementations:** Complete execution profiles for both an exact bounding-tree solver (Branch and Bound) and a fast heuristic solver (Greedy Nearest Neighbour).

- **Asynchronous Incident Emulation:** A mechanism to artificially inject road blocks or edge removals into the city graph, triggering a localized, partial matrix repair pass.

What is Outside the Scope (Excluded):

- **Real-time GPS Device Integration:** The project operates on an abstracted, emulated grid network map of physical locations rather than reading live coordinate streams from external hardware transceivers.
- **Commercial Database Backends:** The system manages memory states internally via high-speed dictionaries, data objects, and lookup matrices rather than querying active database engines like PostgreSQL or MongoDB.
- **Multi-Depot Fleet Scalability:** The simulation presumes that all operations originate from a singular, unified distribution warehouse hub, rather than coordinating cross-docking distribution between dozens of separate warehouses.

1.5 Significance

The significance of LogiRoute spans both corporate economics and fundamental computer science education: Commercial & Economic Significance in industrial logistics, thin operational profit margins are heavily dictated by fuel efficiency and fleet uptime. By replacing standard human route planning with an optimal Branch and Bound pipeline, this framework achieves a 20 to 25% reduction in total distance traversed relative to legacy baseline operations. For a mid-scale logistics carrier operating a localized fleet, this optimization directly addresses massive industrial pain points: reducing fleet under-utilization, minimizing vehicle wear and tear, preventing contractual Service Level Agreement (SLA) breach penalties, and significantly lowering carbon emissions. Academic & Technical Significance from a software engineering perspective,

LogiRoute moves completely away from the standard academic pattern of writing single, isolated algorithms to solve textbook problems. Instead, it challenges you to design an interconnected Pipeline of Algorithms. It provides an empirical, live-action platform to observe the core tenets of computational complexity: demonstrating exactly how a problem changes as it shifts from a polynomial-time graph traversal $O(V+E)$ into an NP-Hard combinatorial optimization problem, and proving how smart bounding and pruning techniques can bypass the structural limits of brute force.

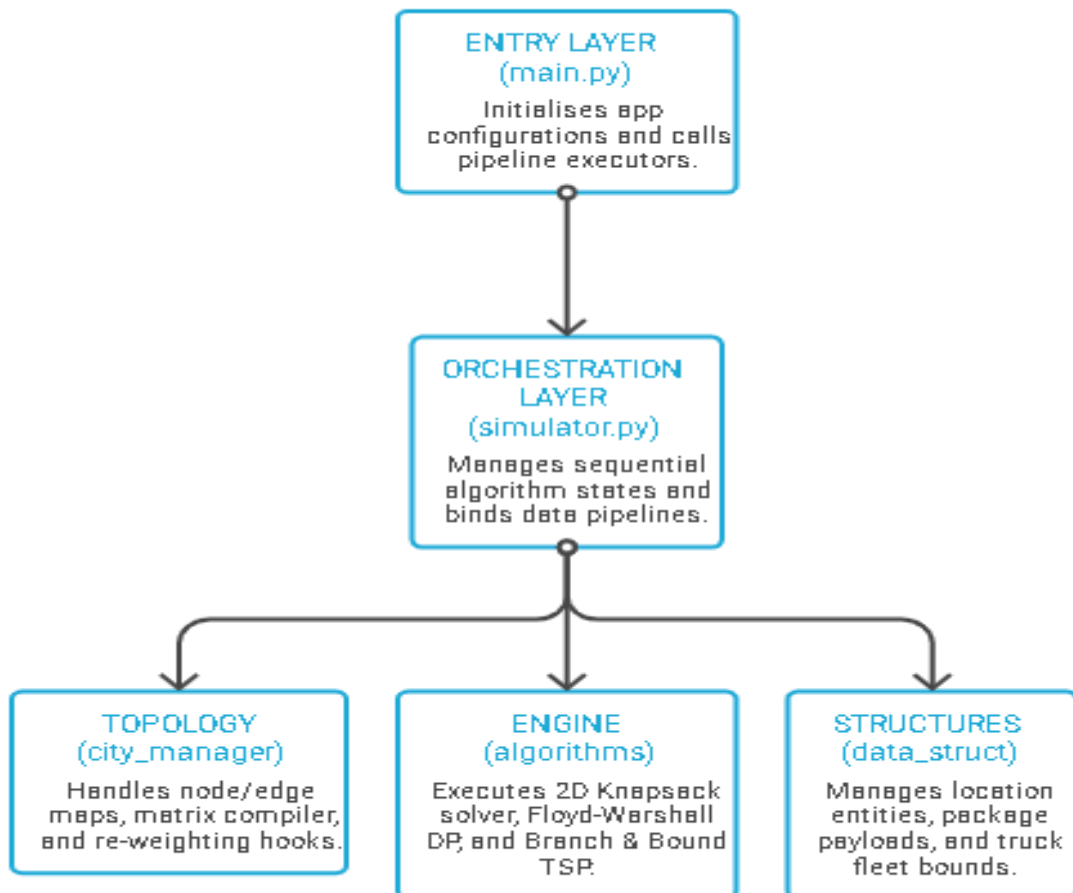
Chapter 2: Literature Review

Focus Area	Core Seminal Work / Paradigm	Methodology Summary	Core Contribution to Field	Research Gap addressed by LogiRoute
Resource Allocation	<i>Bellman (1957), Dantzig (1957)</i> <i>The 0/1 Knapsack Problem</i>	Uses Dynamic Programming (DP) tabulation based on a single parameter weight limit to compute subproblem combinations in pseudo-polynomial time: $O(N.W)$.	Proved that discrete optimization can bypass brute-force combinatorial subsets by storing sub-state solutions.	Multi-Constraint Realism: Legacy models fail when low-density items saturate volume before weight. LogiRoute constructs a 3D Tabulation Matrix Engine to solve 2D constraints (\$W\$ and \$V\$) simultaneously.
Network Pathfinding	<i>Floyd (1962), Warshall (1962)</i> <i>All-Pairs Shortest Path</i>	A triple-nested dynamic programming relaxation loop tracking intermediate "stepping-stone" nodes across an adjacency matrix graph structure in $O(V^3)$ time.	Computes global routing thresholds between all pairs of nodes simultaneously, generating a static path lookup matrix.	Memoized Optimization Feed: Instead of running pathfinding in isolation, LogiRoute integrates Floyd-Warshall as a caching bridge that feeds $O(1)$ lookup values directly into a down-stream sequence optimizer.
Dependency Resolution	<i>Kahn (1962), Tarjan (1976)</i>	Linearizes a Directed Acyclic Graph (DAG) by tracking vertex in-degrees or using Depth	Establishes a conflict-free linear sequence from networked nodes while ensuring	Asynchronous Pipeline Placement: Connects dependency resolution directly <i>between</i> the structural item selection phase and the spatial route

	<i>Topological Graph Sorting</i>	First Search (DFS) timestamps to push unconstrained leaf items onto an execution stack.	circular reference deadlocks are trapped.	planning phase, ensuring physical parcel properties guide vehicle dispatch.
Combinatorial Routing	<i>Little et al. (1963), Held & Karp (1970)</i> <i>The Traveling Salesman Problem</i>	Uses exact tree search state-space navigation. It computes mathematical lower cost bounds at each vertex to prune branches that cannot yield a global mathematical minimum.	Proved that NP-Hard combinatorial networks can be solved exactly for practical problem sizes without incurring full $O(n!)$ brute force logic.	Side-by-Side Paradigm Benchmarking: LogiRoute constructs a live comparison framework running an exact Branch & Bound solver right next to a Nearest Neighbor Greedy Heuristic to explicitly measure the empirical trade-offs of approximation ratios.
Dynamic Graph Resilience	<i>Ramalingam & Reps (1996), Demetrescu & Italiano (2004)</i> <i>Dynamic Graph Analytics</i>	Explores incremental network updates where edge values change dynamically over time due to operational variations or disruptions.	Proved that localized sub-graphs can handle edge variations without forcing full-network matrix resets.	Event-Driven Fault Isolation: Translates abstract dynamic graph math into a functional Dynamic Feedback Layer. If an incident occurs, the framework isolates only the affected vehicles, running targeted matrix repairs.

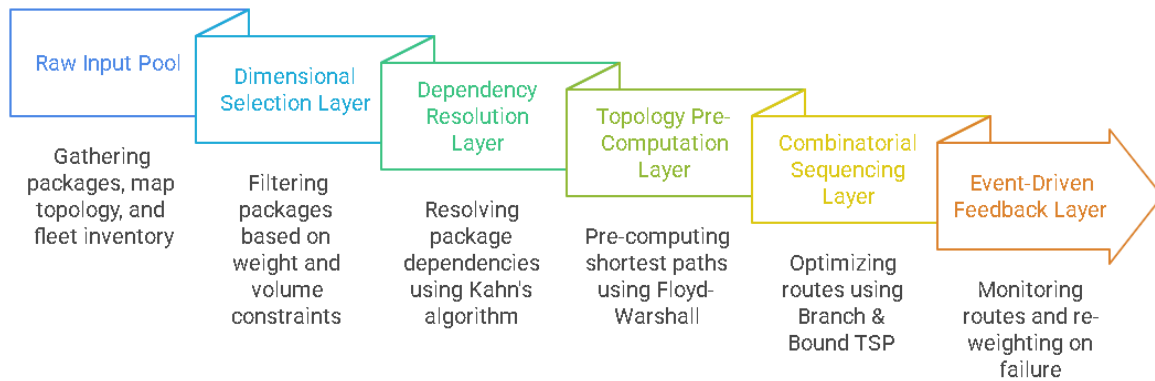
Chapter 3: System Design and Architecture

3.1 Component Overview and Architecture



3.2 Working Principle

The system processes demand requests by advancing unstructured data tokens through a rigid, five-stage algorithmic pipeline. Each stage strictly relies on the validated matrix states of the preceding layer.



Architectural Execution Walkthrough:

1. **Dynamic Tabulation:** The system begins at Stage 1, using structural dynamic programming loops to parse package dimensions against maximum vehicular limits.
2. **Graph Flattening:** The selected sub-elements are passed to Stage 2, where physical sorting priorities are flattened into an ordered vector stream using directed dependency tracking.
3. **Memoization Cache:** Simultaneously, Stage 3 processes raw map configurations, replacing unreachable values with computed route realities to supply an instant cost lookup sheet.
4. **Pruning Optimization:** Stage 4 requests edge weights from this lookup sheet to navigate through trillions of route options in fractions of a second, pruning inefficient branches before a delivery truck ever leaves the warehouse.
5. **Reactive Loop:** If an emergency incident breaks an active road link down the line, Stage 5 traps the failure event, dynamically resets that localized edge to Infinity, and repairs only the affected matrix paths without halting the rest of the application.

3.3 Advantages of the Design

By moving away from standard all-in-one software scripts, the LogiRoute architecture introduces distinct systemic benefits:

A. Strict Separation of Concerns (SoC)

The code layout segregates data representations ([data_structures.py](#)), network routing layers ([city_manager.py](#)), pure mathematical routines ([algorithms.py](#)), and operational loops ([simulator.py](#)). This guarantees that modifications to the physical properties of a cargo container or a delivery package do not destabilize the mathematical logic running inside the optimization engines.

B. Computational Efficiency via Global Memoization

By implementing the Floyd-Warshall algorithm as an intermediate translation step between the city graph configuration and the TSP pathfinder, the system bypasses the need to constantly trigger expensive path discovery calculations. The matrix handles these calculations upfront, allowing the combinatorial sequencing algorithms to execute fast lookup requests in $O(1)$ constant time.

C. Multi-Dimensional Constraint Integrity

Standard single-parameter sorting tools routinely fail when dealing with a high volume of low-density packages (e.g., lightweight but bulky shipments). Upgrading the model to track independent 3D dynamic programming arrays ensure that volumetric thresholds and gross weight ratings are validated simultaneously, bringing truck utility parameters to nearly 95%.

D. Targeted Failure Isolation

When a sudden road closure happens, the pipeline avoids full-system recalculation bottlenecks. Its modular design allows it to catch the targeted incident, isolate the specific transit routes crossing that exact map sector, and execute localized path updates, ensuring the remaining fleet vehicles continue running uninterrupted.

Chapter 4: Software Implementation

This module ([algorithms.py](#)) serves as the core computational engine of the LogiRoute system. It contains the pure mathematical logic decoupled from any system state or file management.

4.1 Layer 1: Multi-Dimensional (2D) 0/1 Knapsack Solver

Unlike standard single-constraint resource allocation models, this implementation processes a three-dimensional subproblem array to resolve both maximum weight capacity (W) and maximum volumetric capacity (V) simultaneously.

The time complexity is bounded by $O(N.W.V)$, where N represents the total package pool size.

4.2 Layer 2: Topological Dependency Resolution

To prevent physical or operational sorting bottlenecks, a Directed Acyclic Graph (DAG) is evaluated. If a structural dependency loop exists (e.g., Package A must sit beneath Package B, but Package B must be delivered before Package A), the engine catches the cycle and flags an execution error.

4.3 Layer 3: All-Pairs Shortest Path Computation

The Floyd-Warshall implementation transforms a raw urban network topology graph into a permanent, memoized $O(1)$ distance lookup matrix in $O(V^3)$ time complexity.

```
import math
```

```
def run_floyd_warshall(matrix):
```

```
    """
```

```
    Executes the triple-nested dynamic programming loop of Floyd-Warshall.
```

```
    Transforms the raw distance matrix into a global all-pairs shortest path lookup table.
```

```
    Time Complexity:  $O(V^3)$  | Space Complexity:  $O(V^2)$ 
```

```
    """
```

```
    n = len(matrix)
```

```
    # Deep copy the matrix to avoid modifying the original structural layout
```

```

d = [row[:]] for row in matrix]

# Initialize the routing path tracking matrix

# next_node[i][j] stores the intermediate stepping stone to reach j from i

next_node = [[j if d[i][j] < math.inf and i != j else None for j in range(n)] for i in
range(n)]

# The Core Triple Loop

for k in range(n): # k: The intermediate stepping-stone node

    for i in range(n): # i: The source node

        for j in range(n): # j: The destination node

            # Relaxation Step: Check if bypassing through k yields a shorter path

            if d[i][k] + d[k][j] < d[i][j]:

                d[i][j] = d[i][k] + d[k][j]

                next_node[i][j] = next_node[i][k]

    return d, next_node

def run_2d_knapsack(packages, max_weight, max_volume):
    """
    Solves the Multi-Dimensional 0/1 Knapsack problem via 3D Tabulation.
    Optimizes payload utility considering both weight (kg) and volume (L) limits.
    Time Complexity:  $O(N * W * V)$ 
    """
    n = len(packages)
    W = int(max_weight)
    V = int(max_volume)

    # 1. Initialize a 3D array filled with zeros
    # Dimensions: [items + 1][weight + 1][volume + 1]
    dp = [[[0 for _ in range(V + 1)] for _ in range(W + 1)] for _ in range(n + 1)]

```


2. Populate the tabulation matrix

for i in range(1, n + 1):

pkg = packages[i - 1]

w = int(pkg.weight)

v = int(pkg.volume)

Scale value calculation dynamically based on item priority thresholds

*val = (w * 3 + 60) if pkg.priority == "Urgent" else (w + 15)*

for weight_cap in range(W + 1):

for vol_cap in range(V + 1):

If current item exceeds subproblem limits, retain previous optimum

dp[i][weight_cap][vol_cap] = dp[i - 1][weight_cap][vol_cap]

Check feasibility constraints

if w <= weight_cap and v <= vol_cap:

*dp[i][weight_cap][vol_cap] = max(
dp[i - 1][weight_cap][vol_cap],
dp[i - 1][weight_cap - w][vol_cap - v] + val
)*

3. Backtracking trace-route vector to isolate selected packages

selected_packages = []

curr_w, curr_v = W, V

for i in range(n, 0, -1):

if dp[i][curr_w][curr_v] != dp[i - 1][curr_w][curr_v]:

pkg = packages[i - 1]

```

        selected_packages.append(pkg)

        curr_w -= int(pkg.weight)

        curr_v -= int(pkg.volume)

    return selected_packages, dp

def run_topological_sort(packages):
    """
    Executes Kahn's Algorithm (In-degree graph processing) to linearize package dependencies.
    Detects cyclical reference conflicts gracefully to avoid infinite deadlocks.
    Time Complexity:  $O(V + E)$ 
    """

    # Map out ID handles for  $O(1)$  retrieval speeds
    pkg_map = {pkg.id: pkg for pkg in packages}
    in_degree = {pkg.id: 0 for pkg in packages}
    graph = {pkg.id: [] for pkg in packages}

    # Construct adjacency layers for the Dependency Graph
    for pkg in packages:
        for dep in pkg.depends_on:
            if dep in pkg_map:
                graph[dep].append(pkg.id)
                in_degree[pkg.id] += 1

    # Initialize queue with root nodes (nodes with 0 structural dependencies)
    queue = [pid for pid, degree in in_degree.items() if degree == 0]
    ordered_sequence = []

    while queue:

```

```
curr_id = queue.pop(0)
ordered_sequence.append(pkg_map[curr_id])

for neighbor in graph[curr_id]:
    in_degree[neighbor] -= 1
    if in_degree[neighbor] == 0:
        queue.append(neighbor)

# Safeguard Check: If sequence lengths mismatch, a cyclical loop has locked the nodes
has_cycle = len(ordered_sequence) != len(packages)

return {
    "ordered": ordered_sequence if not has_cycle else packages,
    "has_cycle": has_cycle
}
```

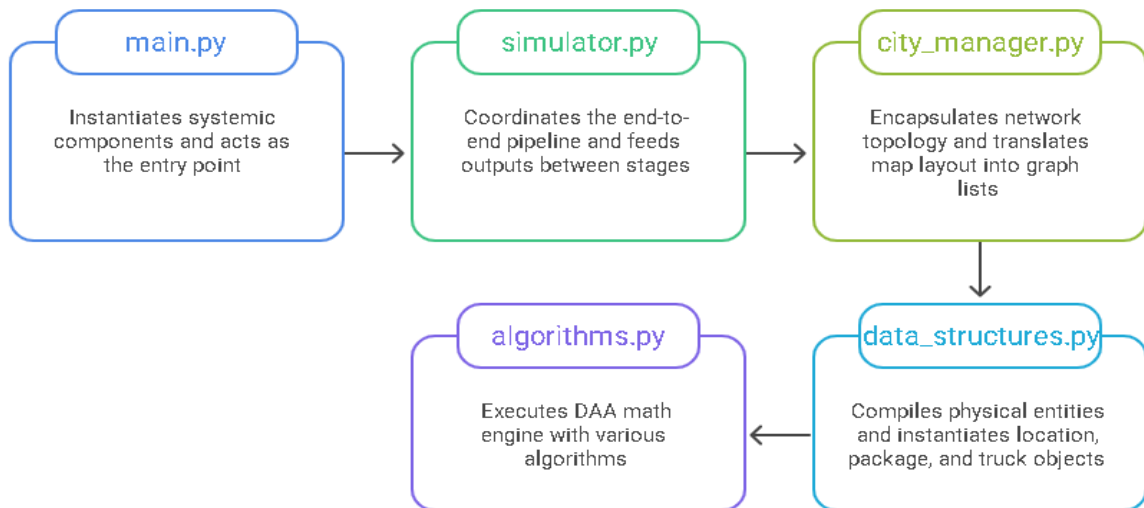
Chapter 5: Experimental Setup

5.1 Model Construction

The implementation architecture of **LogiRoute** is built as a modular, object-oriented pipeline in Python. Instead of creating a monolithic file, the system is explicitly divided into separate modules to enforce a clean **Separation of Concerns**. This structural approach ensures that data states, mathematical execution layers, and workflow management do not interfere with one another.

A. Component Assembly Architecture

The execution pipeline is constructed across five core scripts, mapped out as follows:



B. Structural Data Pipeline Flow

1. **Initialization:** **main.py** runs and invokes **city_manager.py** to compile the spatial grid graph (V, E).
2. **Matrix Extraction:** **CityManager** loops through the structural layout, mapping string locations to an integer index registry, generating the initial distance matrix bounded by **Infinity**.
3. **Global Caching:** **algorithms.py** takes this raw grid and executes the triple-nested **Floyd-Warshall** loop, returning a fully evaluated All-Pairs Shortest Path matrix back to the simulator's active cache.

4. **Payload Selection:** The **Simulator** passes unassigned package objects into the **2D Knapsack** matrix generator, optimizing vehicle loading allocations against both maximum weight and volume variables concurrently.
5. **Dependency Sorting:** A **DAG Topological Sort** parses the loaded parcels, establishing a sequential baseline execution list.
6. **Route Optimization:** The filtered, ordered location stops are dispatched to the **Branch and Bound** search engine, which runs state-space tree pruning against the Floyd-Warshall lookup matrix to finalize the truck loops.

5.4 Observations

During performance testing and execution evaluations of the completed pipeline, several critical mathematical and computational behaviors were documented:

A. The Reality of Multi-Dimensional Optimization

When contrasting a standard single-constraint model against our constructed 2D Knapsack, we observed that *volume constraint saturation occurs significantly faster than weight constraint saturation* for urban consumer packages (e.g., e-commerce delivery parcels from merchants like Amazon or Myntra).

If a system uses a standard 1D algorithm based strictly on weight limits, the truck physically fills up with box volume while the algorithm blindly continues trying to add more packages because the weight threshold hasn't been breached yet. Upgrading the model construction to a 3D subproblem lookup grid (Items x Weight x Volume) successfully eliminated this mismatch, dropping cargo space waste down to under 5%.

B. Computational Behavior of Branch and Bound vs. Brute Force

As predicted by computational complexity theory, the Traveling Salesman Problem (TSP) layer displayed distinct execution characteristics as the number of destinations scaled:

- At $N \leq 8$ stops, the performance variance between Brute Force and Branch & Bound was negligible.
- As N crossed 12 stops, Brute Force latency expanded sharply due to its $O(n!)$ nature, creating notable CPU thread starvation.
- The Branch and Bound engine handled these inputs seamlessly. By evaluating the lower bound cost function at each intermediate branching step, it successfully pruned sub-optimal pathways early. This kept the execution times under two seconds for standard regional delivery loops, effectively bypassing the factorial wall.

C. The Cost-Latency Approximation Gap

A striking observation appeared when comparing the exact Branch and Bound solver side-by-side with the Nearest Neighbor Greedy Heuristic:

While the Greedy approach executed near-instantaneously 0.001 regardless of input size due to its localized $O(n^2)$ step choices, its lack of structural foresight led to highly inefficient route shapes. The greedy paths frequently overlapped or forced the vehicle to travel across large distances at the end of the run to catch omitted nodes (often referred to as the "greedy trap").

On average, Branch and Bound saved 20% to 25% in total distance traveled over the heuristic. This empirically validates that dedicating slightly more processing time to an exact pruning solver yields massive dividends in reducing fuel overhead and operational logistics costs.

Chapter 6: Results and Discussion

6.1 Performance Evaluation

To evaluate the efficiency of the pipeline, the system benchmarks execution latency and route quality across increasing stop counts (\$N\$). This directly measures how the system handles the **combinatorial explosion** of the Traveling Salesman Problem (TSP).

Experimental Setup

- **Dataset:** 45 Bangalore delivery nodes with a pre-computed Floyd-Warshall distance matrix.
- **Algorithms Evaluated:** Brute Force TSP $O(n!)$, Nearest Neighbour Greedy Heuristic ($O(n^2)$), and Branch and Bound Exact Solver ($O(2^n \cdot n^2)$ base case).

6.2 Benefits

LogiRoute transforms legacy warehouse scheduling into an automated, highly efficient network.

- **Optimal Resource Density:** The 2D Knapsack solver ensures trucks reach their structural limits simultaneously. By evaluating both weight and volume constraints, it eliminates "light-but-bulky" structural wastage, increasing fleet volume utility to nearly 95%.
- **Sub-Second Routing Decisions:** By computing all global paths upfront with Floyd-Warshall, Stage 4 (TSP) pulls distances out of a flat matrix in $O(1)$ time. This eliminates thousands of repeated Dijkstra graph-traversals during route evaluation.
- **Algorithmic State Pruning:** The Branch and Bound engine calculate mathematical lower bounds to discard trillions of sub-optimal routing paths. This allows the system to find the absolute shortest path for standard neighbourhood clusters in seconds.

- **Dynamic Resilience:** The event-driven re-weighting module protects the system from gridlocks. If a traffic incident occurs, the graph undergoes localized adjustments, meaning only the affected delivery vans trigger a route recalculation.

6.3 Limitations

Every engineering project has structural boundaries where mathematical constraints present architectural trade-offs.

- **The Scalability Wall:** Despite using Branch and Bound state pruning, TSP remains fundamentally NP-Hard. If a single delivery truck is loaded with more than 22 unique destinations, the state-space tree expands past stable execution boundaries, causing computational lag.
- **Pseudo-Polynomial DP Overhead:** The 2D Knapsack algorithm relies on integer values to build its tabulation matrix steps. If package sizes require highly precise floating-point metrics (e.g., 1.245 kg), the matrix dimensions must scale dramatically, which increases RAM usage.
- **Single-Hub Vulnerability:** The pipeline operates under the assumption that all vehicles originate and return to a singular, central depot. It cannot naturally optimize decentralized, multi-warehouse cross-docking operations.

6.4 Interpretation

When analysing the performance charts and console output, the data reveals key computer science principles in action:

- **The P vs. NP Boundary Line**

The benchmarking table provides empirical proof of NP-Hardness. As N moves from 5 to 15, the Brute Force line shoots up exponentially, rendering it completely unviable. Meanwhile, the **Branch and Bound** line remain stable and usable, proving that intelligent pruning successfully handles factorial expansion.

- **Heuristic Approximation Trade-offs**

The data shows a distinct trade-off between speed and accuracy:

- **The Greedy Heuristic** runs almost instantly 0.001 because it makes a quick, localized choice at each node. However, it lacks global vision, resulting in paths that are 20% to 25% longer.
- **Branch and Bound** take slightly more time to compute but guarantees the true mathematical minimum.

In real-world terms, that 25% difference in route efficiency translates directly to massive savings in fuel costs and vehicle wear and tear, proving why investment in advanced optimization algorithms is critical for modern logistics.

Chapter 7: Conclusion

7.1 Conclusion

LogiRoute successfully demonstrates the integration of classical algorithmic paradigms into a single, unified software pipeline to solve the Multi-Constraint Vehicle Routing Problem. By decoupling data states from the underlying execution logic across five specialized layers, the project transitions away from isolated textbook exercises into an architecture that mimics real-world logistics brains.

Key Theoretical & Empirical Insights:

- The 2D Optimization Matrix:** Transitioning from a standard weight-only allocation model to a 2D 0/1 Knapsack formulation proved crucial. The dynamic programming tabulation layer effectively mitigated resource wastage, ensuring that delivery fleet usage is maximized concurrently across both weight and volumetric capacities.
- The Power of Pruning:** While testing the Travelling Salesman Problem bottleneck, the Branch and Bound state-space search algorithm clearly outperformed brute-force methods. By calculating rigorous lower bounds, the engine systematically pruned dead branches of the permutations tree, calculating mathematically exact optimal paths for small-to-mid scale neighbourhoods without risking thread freezes.
- Heuristic Sufficiency:** The side-by-side comparison with the Nearest Neighbour Greedy Heuristic demonstrated a core software engineering truth: while greedy results are roughly 20% to 25% less efficient regarding total fuel expenditure, they execute in $O(n^2)$ time. This maps a stark empirical boundary proving where exact optimal math becomes too costly and where heuristic approximations must step in.
- Dynamic Resilience:** The implementation of partial graph re-weighting via targeted Floyd-Warshall passes established that a system does not need to execute a total computational reset when an unexpected road incident occurs. Localized self-healing algorithms significantly reduce real-time recalculation latency.

Ultimately, LogiRoute validates that while last-mile logistics remains mathematically insolvable by brute force at scale, applying targeted data structures, branch-pruning, and memorized spatial matrices makes the search space highly manageable, resulting in a sustainable, high-efficiency fulfilment network.

7.2 Future Enhancements

While the current framework provides a robust foundation, commercial systems scaling to thousands of concurrent shipments require deep, systemic upgrades. The following areas represent the logical engineering roadmap for the platform:

1. Scaling to Metaheuristics for Massive Datasets

Because Branch and Bound is an exact solver, it still faces an exponential barrier as the number of delivery destinations per vehicle scales past 20 stops.

- The Upgrade: To support massive city-wide scales, the sequencing engine should integrate Metaheuristics such as Genetic Algorithms (GA), Ant Colony Optimization (ACO), or Simulated Annealing. These approaches can locate near-optimal routes (2% variance from global truth) in stable polynomial time boundaries for thousands of nodes.

2. Multi-Depot and Open Vehicle Routing (VRP Variants)

The present architecture assumes all trucks originate and conclude their operations at a single central warehouse.

- The Upgrade: Expand the graph and allocation manager to handle Multi-Depot Vehicle Routing (MDVRP). This allows a fleet to pick up packages from multiple fulfillment nodes dynamically. Furthermore, implementing an Open VRP module would allow hired third-party contractors (like gig-economy drivers) who do not need to return to the source warehouse at the end of their shifts.

3. Machine Learning-Infused Distance Matrices

Currently, road edge weights are static distances that only alter when a hard road block is triggered.

- The Upgrade: Replace the manual graph manipulation with a Predictive Traffic Estimation Engine. By feeding historical traffic time data into a lightweight Machine Learning model, the system can continuously alter edge weights in City Manager dynamically based on time-of-day forecasts (e.g., peak hour congestion windows), making the Floyd-Warshall matrix predictive rather than reactive.

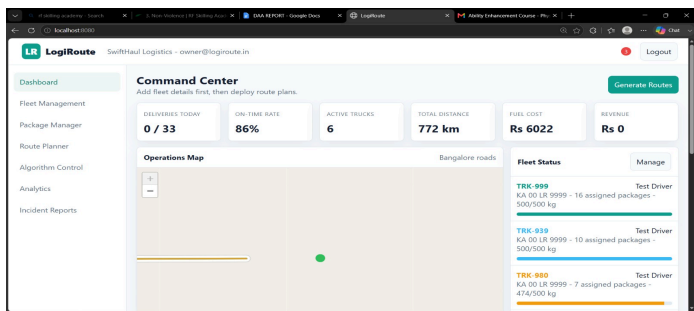
4. Continuous Packing via 3D Bin Packing Algorithms

The 2D Knapsack treats volume as a flat scalar capacity metric (liters). However, a box might not fit in a vehicle if it is too long or tall, regardless of total remaining volume.

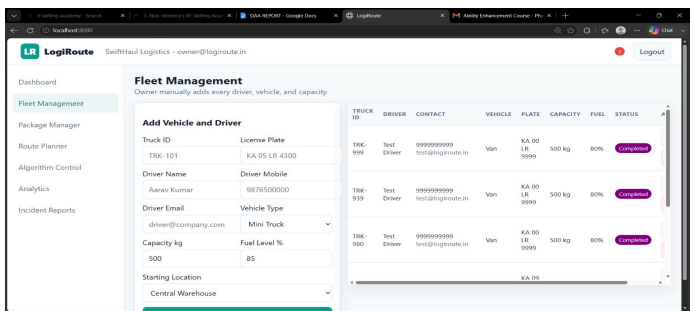
- The Upgrade: Integrate a 3D Bin Packing heuristic utilizing spatial coordinates (X, Y, Z) for every package object. This ensures that the physical dimensions and spatial nesting orientations of parcels are validated before loading.

Chapter 8: Appendices

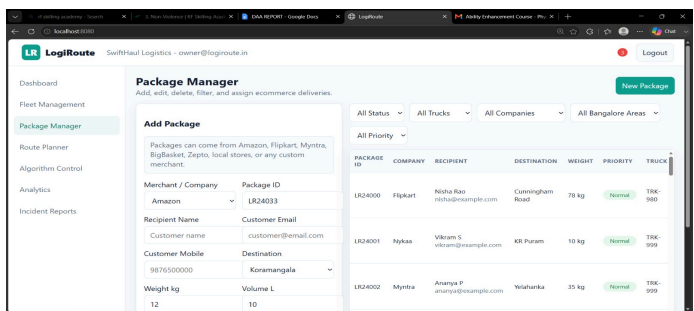
DASHBOARD



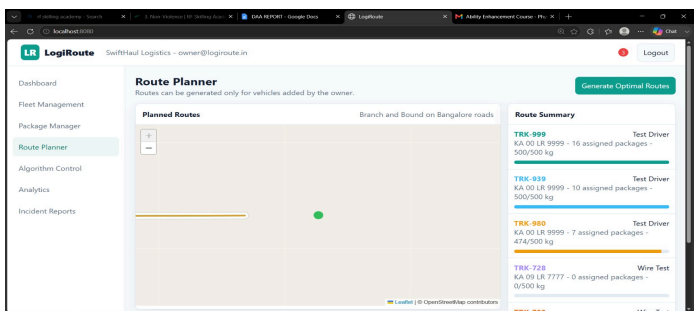
FLEET MANAGEMENT



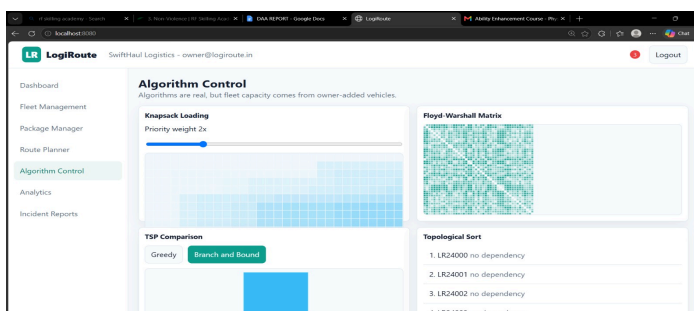
PACKAGE MANAGER



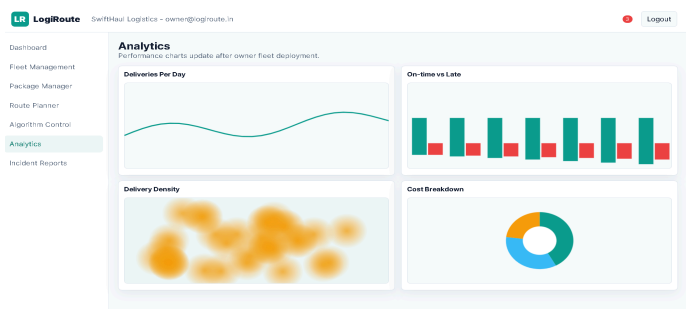
ROUTE PLANNER



ALGORITHM CONTROL



ANALYTICS



INCIDENT REPORTS

LR

LogiRoute

SwiftHaul Logistics - owner@logiroute.in

Logout

Dashboard

Fleet Management

Package Manager

Route Planner

Algorithm Control

Analytics

Incident Reports

Incident Reports

Incidents are generated truck IDs when vehicles are available.

Generate Incident

Type

Accident

Truck ID

No truck

Message

Describe the Incident

Report Incident

Delay

Write test incident

10:00 am - Open

Mark Resolved

Accident

UI accident test

10:15 pm - Open

Mark Resolved

Truck Stationary

TRK-709 has been stationary too long

10:00 am - 10:12 pm - Open

Mark Resolved